

Taller de programación 2025

Unidad 1: Programación estructurada

Comencemos repasando algunas definiciones básicas vistas en CADP.

En Pascal, una **estructura de datos** es un tipo de dato definido por el programador. Una estructura de datos sirve para almacenar varios datos (que guardan alguna relación lógica entre sí) en un solo “paquete”, o sea, en una misma estructura, bajo un único nombre.

Existen cuatro criterios o categorías para clasificarlos.

[1] Si los elementos que componen la estructura son —necesariamente— todos del mismo tipo, entonces la estructura es de tipo **homogénea**. De lo contrario, si los elementos que la componen *pueden* ser de distinto tipo, entonces es **heterogénea**.

[2] Si su tamaño se define cuando se declara la estructura y no puede variar durante la ejecución del programa, entonces es de tipo **estática** (o sea, se calcula cuando se compila el programa y no varía). En cambio, si puede variar con la ejecución del programa, es **dinámica**.

[3] Si para acceder a un elemento particular se debe respetar un orden predeterminado, o sea, siguiendo una secuencia ordenada hasta llegar a donde queremos, entonces la estructura tiene un tipo de acceso **secuencial**. En contraste a esto, si podemos acceder directamente a donde se nos antoje sin necesidad de andar recorriendo los elementos anteriores (por ejemplo, referenciando una posición), entonces la estructura tiene un tipo de acceso **directo**.

[4] Por último, una estructura es **lineal** cuando está formada por 0, 1 o varios elementos que guardan una relación de adyacencia ordenada, donde a cada elemento le sigue y le precede uno solamente. Y es **no lineal** si para un elemento dado pueden existir 0, 1 o más elementos que le suceden y 0, 1 o más elementos que le anteceden.

Con todo esto, ya podemos clasificar a dos estructuras de datos muy importantes: los vectores y las listas.

El **ARRAY** (o arreglo) es una estructura de datos compuesta que permite acceder a cada elemento mediante una variable índice (o algunas variables índices), que referencia(n) la posición del elemento dentro de la estructura de datos. El array puede ser de n dimensiones, es decir que nos permite crear matrices —eso lo vamos a ver en la unidad siguiente, con Java. Pero por ahora, con Pascal, trabajaremos solamente con arreglos de una dimensión, es decir, con vectores (aunque, por costumbre, a veces también los llamamos simplemente arreglos).

Un **VECTOR** es una colección de elementos guardados consecutivamente en la memoria y se pueden referenciar a través de un índice. El vector es una estructura [1] homogénea, [2] estática, [3] de acceso directo y [4] lineal. (En vez de decir que es de acceso directo, también vale decir que es una estructura “indexada”, ya que para acceder a cada elemento de la estructura se debe utilizar una variable índice que es de tipo ordinal).

Una **LISTA** es una colección de nodos. Cada nodo contiene un elemento (que es el valor que se quiere almacenar en la lista), y una dirección de memoria dinámica que indica dónde se encuentra el siguiente nodo de la lista.

Toda lista tiene un nodo inicial.

Ahora bien, a diferencia de los vectores, los nodos que componen una lista pueden no ocupar posiciones contiguas de memoria (dinámica). Es decir, pueden aparecer dispersos en la memoria (dinámica), pero mantienen un orden lógico interno.

En resumen, la lista es una estructura [1] homogénea, [2] dinámica, [3] de acceso secuencial y [4] lineal.

Perfecto: tenemos vectores y listas. Ahora, ¿qué pasa si están desordenados y queremos ordenarlos? Ordenar listas es una tarea compleja, que no vamos a ver acá. Pero ordenar vectores no es tan difícil, y existen varias maneras de hacerlo. Acá vamos a ver dos o tres. Para ello estudiaremos los famosos

Algoritmos de ordenamiento (o *sorting algorithms*, en inglés).

Por lo general, los más fáciles de implementar son los más ineficientes (principalmente en términos de tiempo de ejecución) y los más difíciles de implementar son los más eficientes.

¿Cómo medimos eso? No lo vamos a ver en detalle, pero básicamente la cosa es así:

- Si siempre requiere la misma cantidad de operaciones, independientemente del input, entonces decimos que es de orden constante, y se denota $O(1)$.
- Si va a requerir $\log n$ operaciones (siendo n la cantidad de elementos del input), entonces decimos que es de orden logarítmico, y lo anotamos $O(\log n)$.
- Si es de complejidad lineal, entonces es $O(n)$.
- Si es log-lineal, entonces es $O(n \log n)$.
- Si es cuadrático, es $O(n^2)$.

¿Qué es esa “O” mayúscula? Es la *O*-notation, que se lee *big O notation*¹. El gran *Introduction to Algorithms* (del MIT) en la página 50 nos dice:

***O*-notation**

O-notation characterizes an *upper bound* on the asymptotic behavior of a function. In other words, it says that a function grows *no faster* than a certain rate, based on the highest-order term. Consider, for example, the function $7n^3 + 100n^2 - 20n + 6$. Its highest-order term is $7n^3$, and so we say that this function's rate of growth is n^3 . Because this function grows no faster than n^3 , we can write that it is $O(n^3)$. You might be surprised that we can also write that the function $7n^3 + 100n^2 - 20n + 6$ is $O(n^4)$. Why? Because the function grows more slowly than n^4 , we are correct in saying that it grows no faster. As you might have guessed, this function is also $O(n^5)$, $O(n^6)$, and so on. More generally, it is $O(n^c)$ for any constant $c \geq 3$.

Traducción simplificada: la *O*-notation sirve para indicar la complejidad temporal máxima estimada de un algoritmo en función del tamaño de su input. O sea, no te dice exactamente cuánto va tardar un algoritmo en ejecutarse: te dice, de forma aproximada, cuántas operaciones va a requerir como máximo según el input; te da una estimación de su orden de crecimiento, asegurandote que no va a ir más allá de esa estimación.

Ahora bien, esta notación sirve en todos los casos: tanto para cuando estamos ante el mejor caso posible, ante el caso promedio o ante el peor caso.

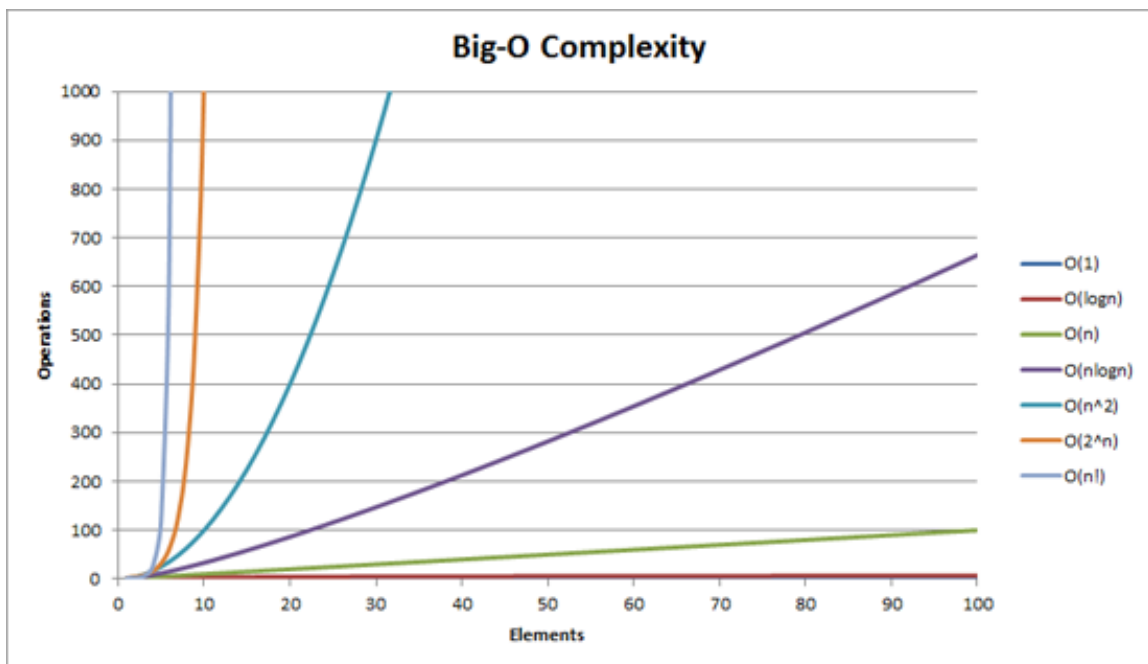
¿Cómo sería eso? Veámoslo con un ejemplo sencillo: la búsqueda simple en un vector.

Supongamos que queremos buscar un elemento en un vector de n elementos que está desordenado. Entonces, lo típico es usar la búsqueda simple: recorrer el vector hasta encontrar el elemento o hasta que se termine el vector. ¿Cuántas operaciones requiere eso? Simplificando un poco las cosas (tal como efectivamente lo hace la *O*-notation), si el elemento era el primero, requiere solamente una operación. Por eso se dice que la búsqueda simple, en el mejor de los casos posibles, tiene una complejidad $O(1)$. Pero si el elemento buscado era el último, requiere n operaciones; con lo cual, en el peor de los casos, tenemos una complejidad $O(n)$.

Ahora bien, normalmente, solemos simplificar todo este lío y hablar siempre de la *O*-notation como la máxima cantidad de operaciones en el peor caso posible. O por lo menos en esta materia lo vamos a ver así.

¹ The letter O was chosen by Bachmann to stand for *Ordnung*, meaning the order of approximation.

Para visualizar un poco todo esto, los distintos crecimientos de las O -notation se pueden representar gráficamente así:



Y algunos de los principales algoritmos de ordenamiento se clasifican así:

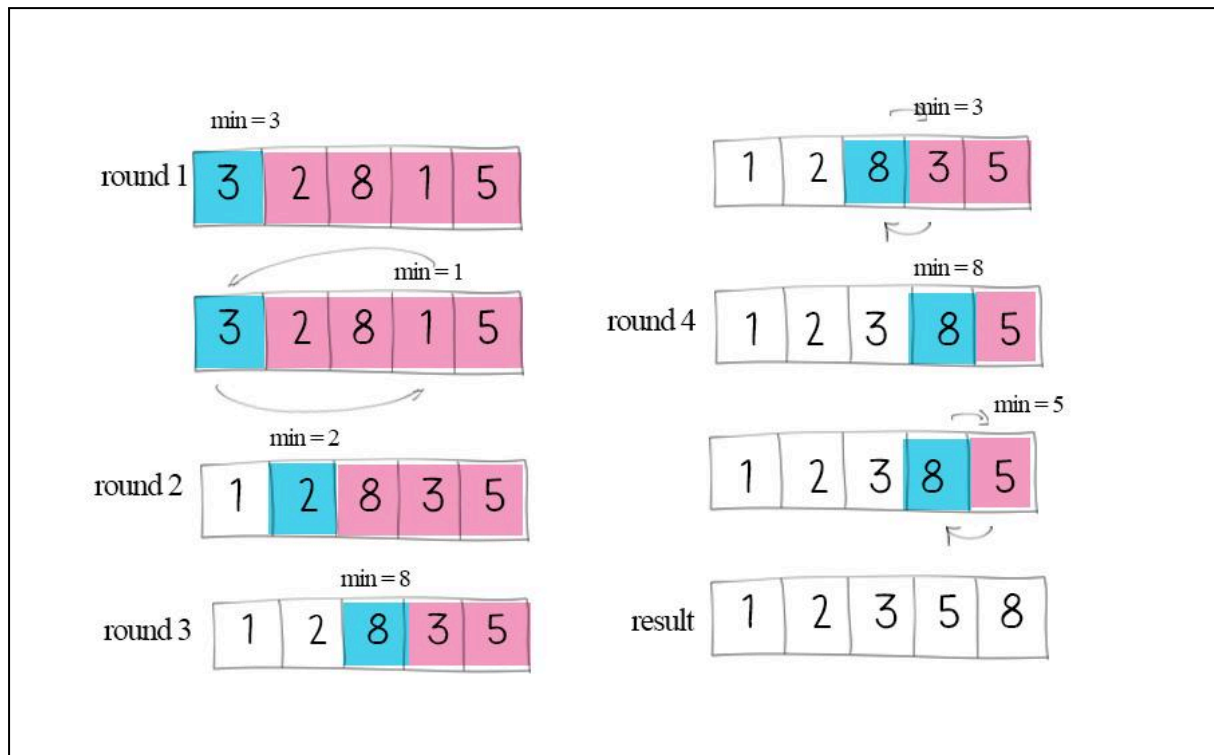
Algoritmo de ordenamiento	Complejidad temporal en el peor caso
Selección (<i>selection sort</i>)	$O(n^2)$
Intercambio (<i>bubble sort</i>)	$O(n^2)$
Inserción (<i>insertion sort</i>)	$O(n^2)$
<i>Heapsort</i>	$O(n \log n)$
<i>Mergesort</i>	$O(n \log n)$
<i>Quicksort</i>	$O(n^2)$

Ok, vayamos, ahora sí, a ver dos algoritmos: el de selección y el de inserción.

Selection sort

Este algoritmo realiza sucesivas pasadas por el vector buscando el elemento más pequeño, y posicionándolo al principio de los que están desordenados. O sea, en la primera pasada recorre todo el vector, y pone el más chico en la posición 1; en la segunda pasada lo mismo, pero en la posición 2; y así siguiendo. Naturalmente, las pasadas son cada vez más chicas: cada nueva pasada debe recorrer una posición menos. Sin embargo, redondeando su complejidad, obtenemos que es $O(n^2)$, ya que debe recorrer (aproximadamente) n veces las n posiciones del vector.

Todo lo dicho puede apreciarse en el siguiente ejemplo, con un vector de 5 elementos (o, lo que es lo mismo, con una dimensión lógica que vale 5):



Otra cosa que podemos notar, es que si el vector tiene n elementos, entonces se realizarán $n - 1$ pasadas, ya que en la última pasada solamente quedan dos elementos, y ambos quedan ordenados. Si hiciéramos n pasadas, la última sería para ordenar un solo elemento, lo cual no tiene sentido.

¿Y el código? Acá va un código casi completo para que lo pruebes. Aunque obviamente lo único que importa es el procedimiento `applySelectionSort()`.

```

program selectionSort;

const dimF = 10; // El tamaño máximo del vector

type vectorEnteros = array [1..dimF] of integer;

procedure cargarVec(var v : vectorEnteros; var dimL : integer);
// Cargalo como quieras

procedure imprimirVec(v : vectorEnteros; dimL : integer);
// Esto es fácil che

// Acá viene lo importante:

procedure applySelectionSort( var v : vectorEnteros; dimL: integer);
var i, j, pos: integer; item : integer;
begin
    // Este for recorre la parte del vector que aún no se ordenó, busca
    // el mínimo, y lo intercambia
    for i := 1 to (dimL - 1) do
        begin
            pos := i;
            // Este for busca el mínimo y guarda su posición en pos
            for j := i + 1 to dimL do
                begin
                    if v[j] < v[pos] then
                        pos := j;
                    end;
                // Acá se intercambia v[i] y v[pos]
                item := v[pos];
                v[pos] := v[i];
                v[i] := item;
            end;
        end;
    end;

var vec : vectorEnteros; dimL : integer;

begin
    cargarVec(vec, dimL);
    applySelectionSort(vec, dimL);
    imprimirVec(vec, dimL);
end.

```

¿Por qué en las variables del procedure separamos i , j y pos de $item$, si son todos integer? Porque las primeras tres siempre serán integer (o un subrango de integer), mientras que el tipo de dato de $item$ va a depender del tipo de dato de vector. Si, por ejemplo, tenemos un vector de reales, entonces $item$ será un real.

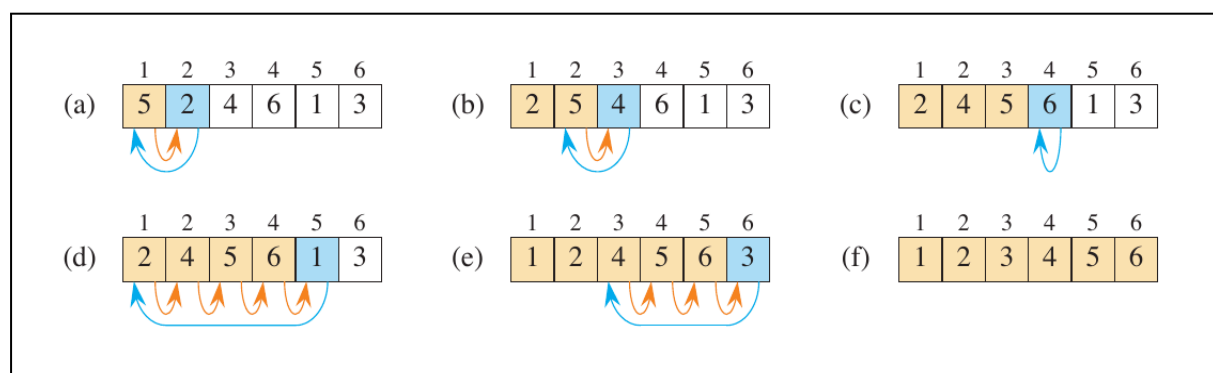
Consideraciones finales sobre *selection sort*:

- Su complejidad temporal en el peor caso es $O(n^2)$.
- Es fácil de escribir.
- La memoria que utiliza durante la ejecución es simplemente el vector y un par de variables auxiliares.
- No requiere estructuras auxiliares.
- Desventaja: requiere el mismo tiempo si los datos ya están ordenados, si están al azar, si se encuentran en el orden exactamente inverso al que yo los quiero tener. Es decir, en todos los casos, su complejidad temporal es $O(n^2)$.

Insertion sort

Si en *selection sort* la idea es ir “decrementando la parte desordenada”, acá la idea es ir “incrementando la parte ordenada”. La estrategia es ir expandiendo el conjunto de los elementos que están ordenados. O sea: empezamos con los primeros dos elementos, y los ordenamos. Después miramos el tercero, y nos fijamos si está desordenado con respecto a los anteriores. Si no está bien, lo ubicamos en su lugar. Con eso tenemos los tres primeros elementos ordenados. Después hacemos lo mismo con el cuarto y con los que resten.

Acá va un ejemplo con n o $dimL = 6$:



Un programa casi completo podría ser así:

```

program insertionSort;

// Mismos tipos de datos y módulos que antes

procedure applyInsertionSort(var v : vectorEnteros; dimL : integer);
var i, j : integer; actual : integer;
begin
    for i := 2 to dimL do
        begin
            actual := v[i]; // Actual es el elemento que se va a insertar en
// la posición correcta
            j := i - 1;
            // Este while busca el lugar (y hace lugar) para la variable
'actual'
            while ((j > 0) and (v[j] > actual)) do
                begin
                    v[j + 1] := v[j];
                    j := j - 1;
                end;
            v[j + 1] := actual;
        end;
    end;

var vec : vectorEnteros; dimL : integer;

begin
    cargarVec(vec, dimL);
    applyInsertionSort(vec, dimL);
    imprimirVec(vec, dimL);
end.

```

Al igual que antes, separamos i y j de actual, ya que las primeras dos siempre serán integer, mientras que actual será del tipo de dato que sea el vector. Pero ojo: LO RECOMENDABLE ES NO USAR SUBRANGO con i y j en este algoritmo, porque si el subrango no incluye el 0, y en algún momento j se va a 0, eso puede causar algún error en la ejecución.

Consideraciones finales sobre *insertion sort*:

- Su complejidad temporal en el peor caso es $O(n^2)$.
- No es tan fácil de escribir como el *selection sort*.

- La memoria utilizada durante la ejecución es, al igual que antes, el vector y algunas variables auxiliares.
- No requiere estructuras auxiliares.
- La gran diferencia y ventaja con respecto al anterior, es que no tarda siempre el mismo tiempo, porque si bien siempre tiene que realizar la misma cantidad de pasadas (a saber, $n - 1$ pasadas), si el elemento que se quiere ordenar (= la variable `actual`) ya está ordenado, no se ejecutará el `while()`, con lo cual podemos considerar que requerirá únicamente una operación en esa pasada. Por lo tanto, se considera que si el vector ya está ordenado de menor a mayor (que es el mejor caso posible), la complejidad temporal de este algoritmo es $O(n)$; y si está ordenado de mayor a menor (lo cual es el peor caso posible) es $O(n^2)$.

En palabras de la cátedra:

Si los datos están ordenados de menor a mayor el algoritmo solo hace comparaciones, por lo tanto, es de orden (N). Si los datos están ordenados de mayor a menor el algoritmo hace todas las comparaciones y todos los intercambios, por lo tanto, es de orden (N^2) comparaciones.

Recursión

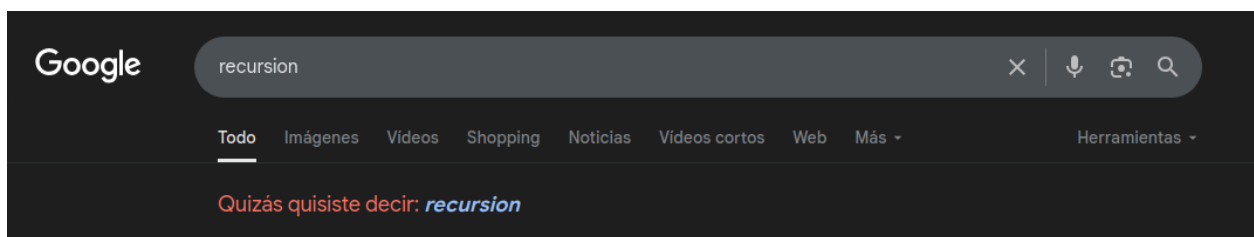
La recursividad es una técnica de resolución de problemas que consiste en dividir un problema en instancias más pequeñas del mismo problema (también llamados subproblemas) hasta que obtengamos un subproblema lo suficientemente pequeño que tenga una solución trivial o directa.

La recursividad consiste en resolver un problema por medio de un módulo (procedimientos o funciones) que se llama a sí mismo, evitando el uso de bucles y otros iteradores.

Cuando el problema se va achicando, llega a un punto que no puede achicarse más. Esa instancia se denomina *caso base*.

Hay problemas en los cuales debe realizarse alguna tarea cuando se alcanza el caso base y otros que no. Hay problemas que pueden tener más de un caso base.

DICHO EN CRIOLLO: *un módulo recursivo, es cuando un módulo se llama a sí mismo*. Por eso, los muchachos de Google, que son muy pillos, hicieron esto:



Comencemos a comprender este asunto con el ejemplo más clásico y más claro y más ejemplo de todos: el factorial.

El factorial de un entero no negativo n , básicamente se define como la multiplicación de todos los números entre 1 y ese número (excepto el 0, que es 1). Siguiendo esta definición, obtenemos una tabla de este estilo:

$0! = 1$ (el caso especial, establecido por definición)

$1! = 1$

$2! = 1 * 2 = 2$

$3! = 1 * 2 * 3 = 6$

$4! = 1 * 2 * 3 * 4 = 24$

Y así siguiendo.

Obviamente, podemos calcularlo de forma iterativa. Por ejemplo así:

```
program ejemploFactorialIterativo;

function factorial(n : integer) : integer;
var i, resultado : integer;
begin
    resultado := 1;
    if (n > 1) then
    begin
        for i := 2 to n do
            resultado := resultado * i;
        end;
        factorial := resultado;
    end;
end;

var n : integer;

begin
    write('Ingrese un entero no negativo: '); readln(n);
    writeln('El factorial de ', n, ' es ', factorial(n));
end.
```

Ahora bien, también podemos entender a los factoriales de forma recursiva, y decir que cada número factorial es la multiplicación de ese número, por el factorial del número que le antecede (hasta llegar a 1, que es 1). Desde este punto de vista, la tabla nos quedaría así:

0! es el caso especial, que vale 1.

1! es el caso base que vale 1.

$$2! = 2 * (2-1)! = 2 * (1)! = 2 * 1 = 2$$

$$3! = 3 * (3-1)! = 3 * (2)! = 3 * 2 = 6$$

$$4! = 4 * (4-1)! = 4 * (3)! = 4 * 6 = 24$$

Entonces, una expresión de este tipo, completamente desarrollada, sería así:

$$4! = 4 * 3!$$

$$4 * 3! = 4 * 3 * 2!$$

$$4 * 3 * 2! = 4 * 3 * 2 * 1!$$

$$4 * 3 * 2 * 1! = 4 * 3 * 2 * 1$$

$$= 4 * 3 * 2 * 1$$

$$= 4 * 3 * 2$$

$$= 4 * 6$$

$$= 24$$

¿Por qué está escrito de esa forma rara? Porque así opera —aproximadamente— una función recursiva. Veámoslo en acción en el siguiente código:

```
program ejemploFactorialRecursivo;

function factorial(n : integer) : integer;
begin
    if (n < 2) then
        factorial := 1
    else
        factorial := n * factorial(n - 1);
    end;
end;

var n : integer;

begin
    write('Ingrese un entero no negativo: '); readln(n);
    writeln('El factorial de ', n, ' es ', factorial(n));
end.
```

Acá tenemos una función recursiva que

- devuelve 1 cuando se ingresa el caso especial 0, o cuando se llega al caso base 1 y
- devuelve $n * (n - 1)!$ cuando $n > 1$

Entonces, si uno ingresa un número (por ejemplo el 4), la función primero va a hacer muchos llamados a sí misma (hasta llegar al caso base), y después se va a empezar a responder a sí misma. El recorrido sería más o menos así:

$\text{factorial}(4) = 4 * \text{factorial}(3)$

$\text{factorial}(3) = 3 * \text{factorial}(2)$

$\text{factorial}(2) = 2 * \text{factorial}(1)$

$\text{factorial}(1) = 1 \rightarrow \text{Caso base}$

$\text{factorial}(2) = 2 * 1 = 2$

$\text{factorial}(3) = 3 * 2 = 6$

$\text{factorial}(4) = 4 * 6 = 24$

¿En vez de una función, podemos hacer un *procedure* recursivo? Sí, también se puede. Por ejemplo, para imprimir una lista (donde el dato es simplemente un integer) sería así:

```
procedure imprimir (pri : lista);
begin
    // Condición de corte: cuando se termina la lista
    if (pri <> nil) then
    begin
        // Acá imprime efectivamente el dato del nodo actual
        writeln(pri^.dato);
        // Avanza al siguiente nodo
        pri := pri^.sig;
        // Va a realizar el mismo procedimiento con ese siguiente nodo
        imprimir(pri);
    end;
end;
```

Esa versión funciona bien, pero hay otras dos que no:

```
// Imprime la lista, pero deja pri en nil en el programa principal
procedure imprimir2 (var pri : lista);
begin
    if (pri <> nil) then
    begin
        writeln(pri^.dato);
        pri := pri^.sig;
        imprimir2(pri);
    end;
end;
```

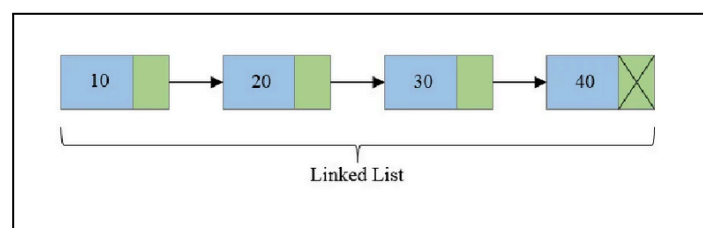
```
// Dos problemas: [1] no imprime el primer elemento; [2] crashea,
porque cuando llega al último nodo, avanza un nodo más, pri queda en
nil, y a pesar de eso se intenta hacer writeln(pri^.dato)
procedure imprimir3 (pri : lista);
begin
    if (pri <> nil) then
    begin
        pri := pri^.sig;
        imprimir3(pri);
        writeln(pri^.dato);
    end;
end;
```

Árboles

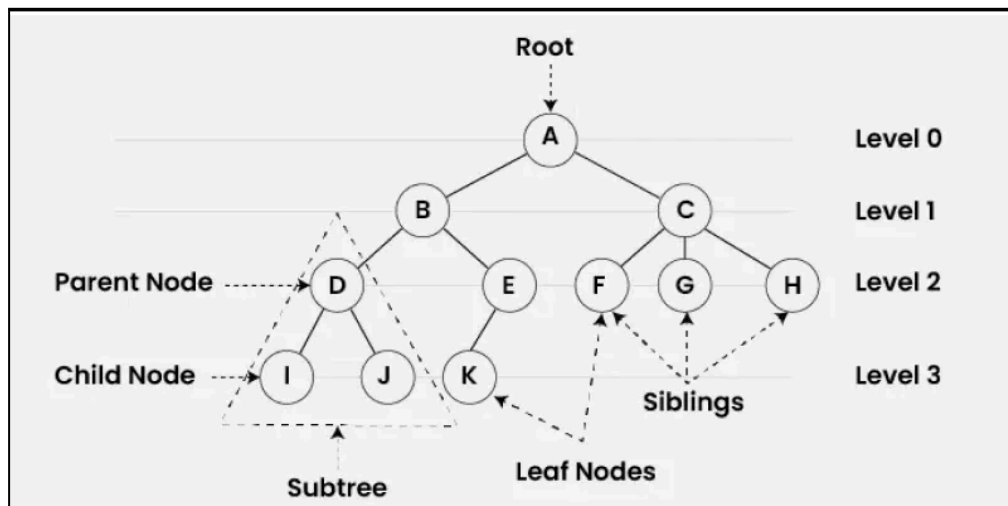
Hasta ahora, con vectores y listas parecía que la vida estaba resuelta. Pero no. La verdad que no. Todavía hay muchas cosas que son difíciles de hacer con esas estructuras de datos, o que no son tan eficientes. Para eso vinieron a rescatarnos, como siempre, los árboles.

Un árbol es como una lista, pero no lineal. O sea: es una colección de nodos, pero en vez de estar uno atrás del otro en fila, están distribuidos de una forma que se asemeja a un árbol dado vuelta. Así que ahora, cada nodo no sólo tiene un nodo siguiente: puede tener dos o más, dependiendo del tipo de árbol con el que estemos trabajando.

Gráficamente, si las listas enlazadas se veían así:



Ahora, un árbol se ve así:



Ahora el primer nodo se llama raíz, los del medio simplemente nodos, y los últimos nodos (es decir, aquellos que no tienen hijos), se llaman hojas.

El árbol es un tipo de dato ideal para representar, entre otras cosas, árboles genealógicos. De hecho, cuando hablamos de él, utilizamos esa terminología: no decimos que cada nodo tiene un siguiente a la izquierda o un siguiente a la derecha. Cada nodo tiene (o puede tener), un hijo izquierdo y un hijo derecho. Por ejemplo: en la imagen, el nodo A tiene dos hijos (B y C), donde B y C son hermanos, y tienen como padre a A.

Pero acá no nos vamos a meter con la familia. Acá los vamos a usar para ordenar datos de forma súper eficiente. ¿Por qué? Porque resulta que hay algunos tipos de árboles que son perfectos para realizar algunas operaciones básicas (como buscar o insertar) si están ordenados, ya que su implementación es relativamente sencilla y muy eficiente.

Acá vamos a trabajar con los árboles binarios (es decir, árboles donde cada nodo tiene dos hijos). Más específicamente, con árboles binarios de búsqueda (*binary search tree*).

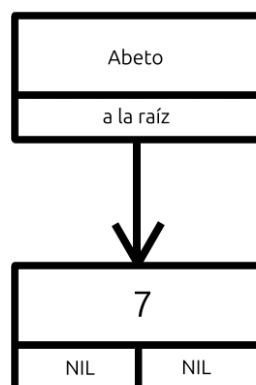
¿Cómo son estos ABB? Típicamente se organizan así: si un nodo es menor que el actual, va para la izquierda, si es mayor, para la derecha. (También podrían ser exactamente al revés (depende de la ideología de cada uno), pero normalmente los vamos a ver así).

Pongamos un ejemplo: tenemos un árbol llamado Abeto que está vacío. ¿Cómo queda si le agrego los nodos con los valores 7, 5, 6, 2, 3, 8?

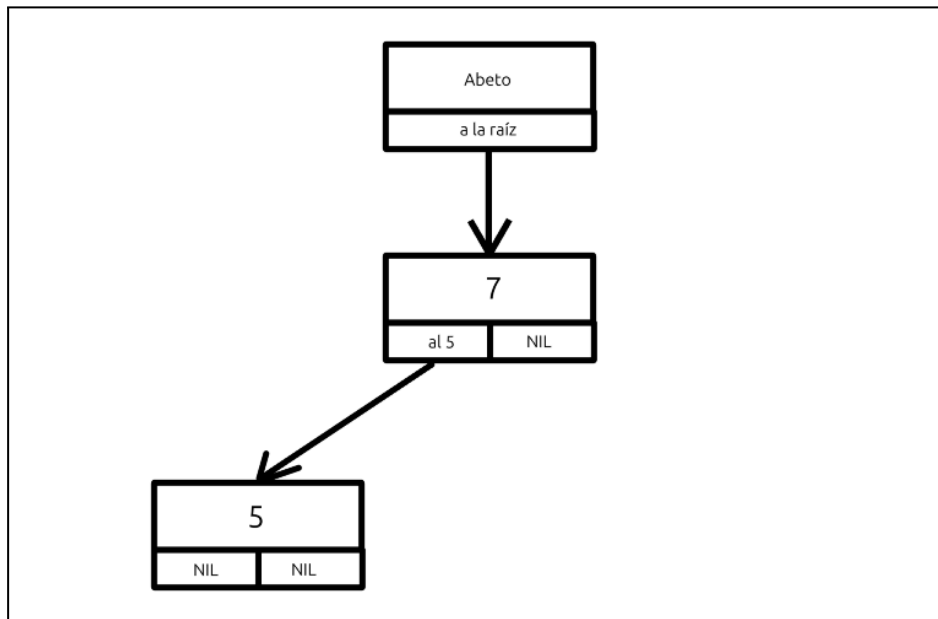
Vayamos paso a paso. Como ya aprendimos en CADP, antes de cargar una lista, hay que crearla. Una lista se crea así: $L := \text{nil}$. Con los árboles pasa lo mismo: antes de cargarlos, hay que crearlos (o inicializarlos, como prefieras decirlo). Entonces, si nuestro árbol se llama Abeto, tenemos que hacer esto: $\text{Abeto} := \text{nil}$.



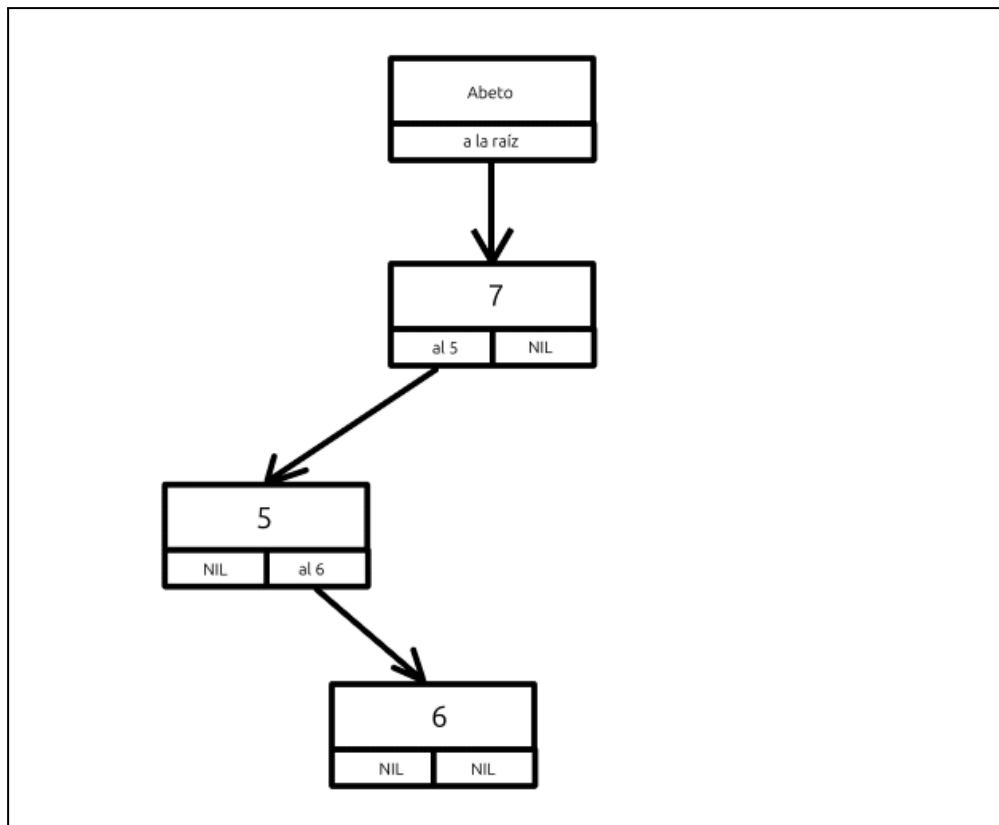
Ahora agregamos el 7. Por ser el primer nodo que agregamos, este nodo será la raíz, y como estamos en un *binary search tree* y no vamos a andar eliminando nodos ni haciendo cosas raras, agreguemos lo que agreguemos, el 7 (por ser el primer nodo agregado) seguirá siendo la raíz.



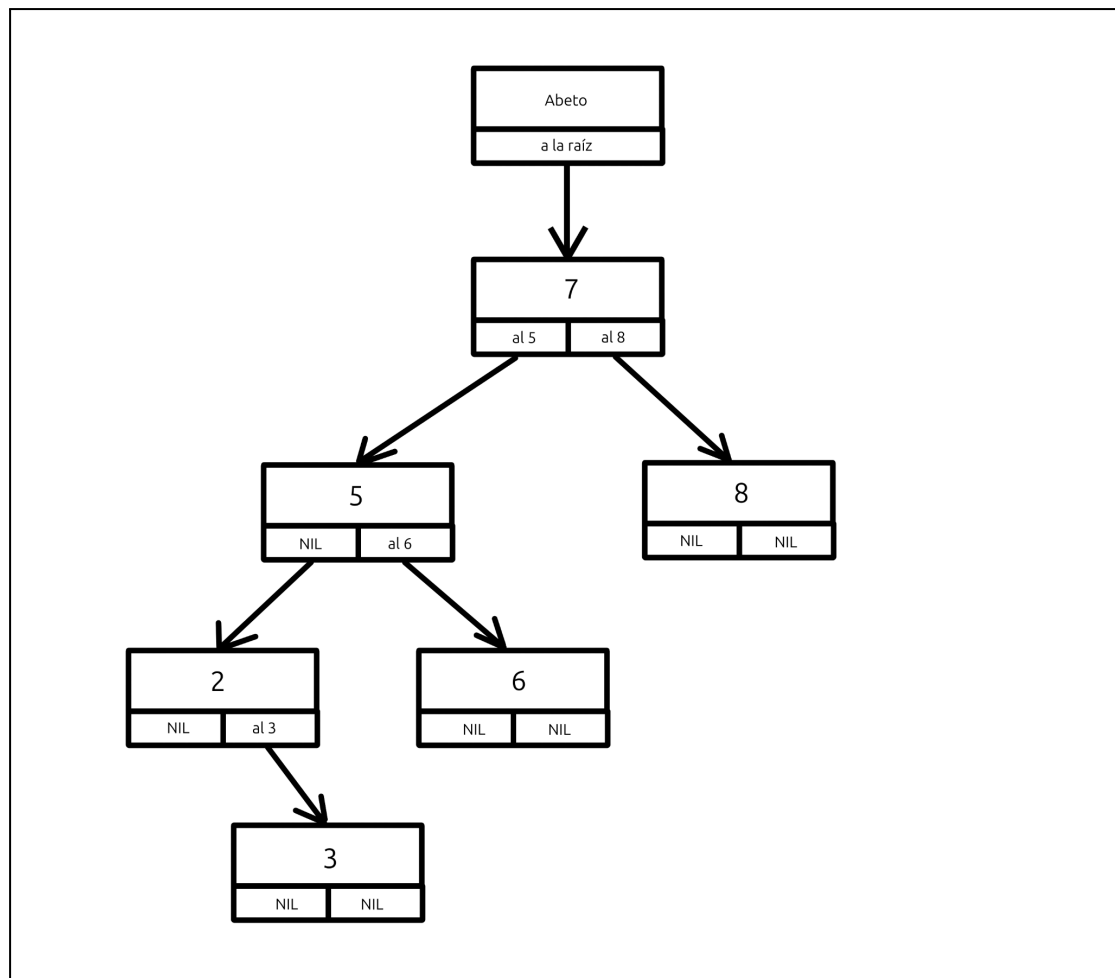
Ahora vamos con el 5. Como el árbol ya no está vacío, tenemos que preguntar: ¿este nodo es menor o mayor al que ya está? Como es menor, lo mandamos a la izquierda, y nos queda así:



Ahora viene el 6. Entonces, empezamos desde la raíz, y preguntamos ¿mayor o menor? Es menor a 7, así que damos un paso a la izquierda. ¿Mayor o menor? Es mayor a 5, así que ahora a la derecha. Y como ya no hay más, lo agregamos:



Siguiendo con esa lógica, si agregamos 2, 3 y 8 nos quedaría así:



Seguramente te estarás preguntando: ¿y el orden? ¿Dónde está el orden? Está en que, para cada nodo, su nodo izquierdo es menor, y su nodo derecho es mayor. Por ello, el orden aparece cuando uno recorre el árbol de cierta manera en particular, que la llamaremos *inorden*. Este algoritmo recursivo consta de tres pasos:

- 1. Visitar subárbol izquierdo: se llama recursivamente a la función de recorrido inorden en el hijo izquierdo del nodo actual.
- 2. Procesar el nodo raíz: una vez que el subárbol izquierdo ha sido completamente recorrido, se procesa el nodo actual (por ejemplo, se imprime su valor).
- 3. Visitar subárbol derecho: finalmente, se llama recursivamente a la función de recorrido inorden en el hijo derecho del nodo actual.

Fijate que si seguís esos pasos, vas a procesar los nodos así: 2, 3, 5, 6, 7 y 8.

Además, una ventaja de estos árboles es que el elemento mínimo es el nodo que está más a la izquierda, y el máximo es el que está más a la derecha. No hace falta recorrer y comparar todos los nodos para obtener esta información.

Ok. ¿Y el código? Primero vayamos al tipo de dato.

Pensemos que un árbol es como una lista, pero en vez de tener un siguiente, tendrá dos, y los llamaremos hijo izquierdo e hijo derecho (para abreviar, HI y HD). Entonces para el tipo de dato, obtenemos esto:

```
type
  arbol = ^nodo;
  nodo = record
    dato : integer;
    HI : arbol;
    HD : arbol;
  end;
```

Ahora vamos con el código completo:

```
program ejemploAbeto;

type // Lo de recién

procedure agregarNodo(var a : arbol; num : integer);
var nuevo : arbol;
begin
  if (a = nil) then // Si ya encontré el lugar para agregarlo, creo el
nodo y lo agrego
  begin
    new(nuevo);
    nuevo^.dato := num;
    nuevo^.HI := nil;
    nuevo^.HD := nil;
    a := nuevo;
  end
  else // Si no, sigo buscando el lugar para agregarlo
  begin
    if (num <= a^.dato) then
      agregarNodo(a^.HI, num)
    else
      agregarNodo(a^.HD, num);
    end;
  end;
end;
```

```

procedure cargarArbol(var a : arbol);
var num : integer;
begin
    write('Ingrese un número: '); readln(num);
    while (num <> 0) do
        begin
            agregarNodo(a, num);
            write('Ingrese un número: '); readln(num);
        end;
    end;

procedure imprimirINORDEN(a : arbol);
begin
    if (a <> nil) then
        begin
            imprimirINORDEN(a^.HI); // Visitar subárbol izquierdo
            write(a^.dato, ' ');    // Procesar el nodo raíz
            imprimirINORDEN(a^.HD); // Visitar subárbol derecho
        end;
    end;

var Abeto : arbol;

begin
    Abeto := nil;
    cargarArbol(Abeto);
    write('Así quedó el árbol inorden: ');
    imprimirINORDEN(Abeto);
end.

```

Y si usamos los valores del ejemplo, nos da lo siguiente:

```

Ingrese un número: 7
Ingrese un número: 5
Ingrese un número: 6
Ingrese un número: 2
Ingrese un número: 3
Ingrese un número: 8
Ingrese un número: 0
Así quedó el árbol inorden: 2 3 5 6 7 8

```

Perfect. ¿Y con preorden y postorden? El código es casi igual:

```
procedure imprimirPREORDEN(a : arbol);
begin
    if (a <> nil) then
    begin
        write(a^.dato, ' ');
        imprimirPREORDEN(a^.HI);
        imprimirPREORDEN(a^.HD);
    end;
end;

procedure imprimirPOSTORDEN(a : arbol);
begin
    if (a <> nil) then
    begin
        imprimirPOSTORDEN(a^.HI);
        imprimirPOSTORDEN(a^.HD);
        write(a^.dato, ' ');
    end;
end;
```

Pero el resultado cambia. Siguiendo el mismo ejemplo de antes, nos daría este resultado:

```
-----Así quedó el árbol inorden: 2 3 5 6 7 8
-Así quedó el árbol en preorden: 7 5 2 3 6 8
Así quedó el árbol en postorden: 3 2 6 5 8 7
```

El complejo arte de interpretar bien las consignas

Cuestiones básicas:

- Si la consigna no dice que la estructura que usamos (como por ejemplo, un vector o una lista) tiene que estar ordenada, y en ningún momento se pide hacer algo que requiera de algún orden, entonces no hay que complicarse con eso. Con cargar esa estructura de la manera más sencilla y rápida va a estar bien.
- Si la estructura que necesito para almacenar datos, tiene un límite máximo, probablemente convenga usar un vector (pero no es obligatorio, hay que analizar en

cada caso si es conveniente o no). Si no tengo un límite máximo, no puedo usar un vector. Puedo usar una lista o un árbol.

- Cuando dice “crear un módulo que retorne x dato”, por lo general se refiere a que ese dato llegue al programa principal, o al módulo que lo llamó. Y aunque no diga “imprimir”, está bueno ir imprimiendo eso que se retorna. Si no, no sabríamos nada de lo que está haciendo el programa.

Ahora veamos algunos ejemplos concretos que tienen ciertas frases típicas, empezando con la consigna 2 de la clase 5.

Una agencia dedicada a la venta de autos ha organizado su stock y tiene la información de los autos en venta. Implementar un programa que:

a) Genere la información de los autos (patente, año de fabricación (2015..2024), marca, color y modelo, finalizando con marca 'MMM') y los almacene en dos estructuras de datos:

i. Una estructura eficiente para la búsqueda por patente.

ii. Una estructura eficiente para la búsqueda por marca. Para cada marca se deben almacenar juntas las patentes y colores de los autos pertenecientes a ella.

Primero analicemos el pedido principal, y los tipos de datos a utilizar. Claramente, lo que dice el inciso a) es que la información se va a leer/generar en un registro de este tipo:

```
const
  ini_anio = 2015;
  fin_anio = 2024;

type
  sub_anios = ini_anio..fin_anio;

  auto = record
    patente : integer;
    anio : sub_anios;
    marca : string;
    color : string;
    modelo : string;
  end;
```

Lo que indica el inciso a) i) con “una estructura eficiente para la búsqueda por ...”, es que tiene que ser un árbol. Y como dice “para la búsqueda por patente”, se refiere a un árbol de autos ordenados según su patente.

Así que, en este caso, el dato del árbol es igual al tipo de dato que leemos.

```
arbolPat = ^nodoPat;  
nodoPat = record // Esto es para el árbol ordenado por patentes  
    dato : auto;  
    HI : arbolPat;  
    HD : arbolPat;  
end;
```

Pero en el siguiente caso no. Cuando el inciso a) ii) dice “Una estructura eficiente para la búsqueda por marca. Para cada marca se deben almacenar juntas las patentes y colores de los autos pertenecientes a ella”, espera que hagamos un árbol donde cada nodo tiene una marca, y una lista con todos los autos de esa marca, de los cuales solamente anotamos su patente y su color. Para eso, hay que seguir creando tipos de datos en *type*, y queda más o menos así:

```
datoLista = record  
    patente : integer;  
    color : string;  
end;  
  
lista = ^nodo;  
nodo = record  
    dato : datoLista;  
    sig : lista;  
end;  
  
datoMarc = record  
    marca : string;  
    l : lista;  
end;  
  
arbolMarc = ^nodoMarc;  
nodoMarc = record // Esto es para el árbol ordenado por marcas  
    dato : datoMarc;  
    HI : arbolMarc;  
    HD : arbolMarc;  
end;
```

Ahora sigamos con el resto de los puntos:

b) Invoque a un módulo que reciba la estructura generada en a) i) y una marca y retorne la cantidad de autos de dicha marca que posee la agencia.

Hay que hacer una función (ya que solo devuelve un tipo de dato simple, un integer), que recibe dos cosas: [1] el árbol de autos ordenado por patente, y [2] una marca, y retorna una sola cosa: cuántos autos hay de esa marca. Y como el árbol está ordenado por patente, el orden no nos sirve para hacer más eficiente el conteo. Así que hay que recorrer todo el árbol e ir contando en cada nodo si la marca es o no es la que buscamos.

Una posibilidad es hacerlo así:

```
function cantMarca(aPat : arbolPat; m : string) : integer;
begin
    if (aPat = nil) then
        cantMarca := 0
    else
        begin
            if (aPat^.dato.marca = m) then
                cantMarca := 1 + cantMarca(aPat^.HI, m) + cantMarca(aPat^.HD, m)
            else
                cantMarca := cantMarca(aPat^.HI, m) + cantMarca(aPat^.HD, m);
            end;
        end;
end;
```

c) Invoque a un módulo que reciba la estructura generado en a) ii) y una marca y retorne la cantidad de autos de dicha marca que posee la agencia.

Misma tarea, pero con el otro árbol que habíamos hecho. Ahora bien: este “pequeño cambio” implica muchos cambios. Entre otras cosas, ahora podemos (o mejor dicho, DEBEMOS) aprovechar el orden. Pensemos un poco.

Tenemos un árbol ordenado por marcas, y de cada marca tenemos, en una lista, los autos de esa marca. Por lo tanto, para saber cuántos autos hay de x marca, tenemos que llegar a esa marca, y contar cuántos elementos tiene la lista (con una función auxiliar que se encargue de eso). Y ya está, no hay que seguir recorriendo el árbol, porque no va a haber otro nodo con la misma marca. Es llegar a ese nodo, y contar la lista. Y en caso de que no lleguemos, porque la marca no existe, obviamente hay que retornar 0.

¿Cómo quedaría el código? Podría ser así:

```

function cantMarca2(aMarc : arbolMarc; m : string) : integer;
begin
    if (aMarc = nil) then
        cantMarca2 := 0
    else
        begin
            if (m = aMarc^.dato.marca) then
                cantMarca2 := contarLista(aMarc^.dato.l)
            else
                begin
                    if (m < aMarc^.dato.marca) then
                        cantMarca2 := cantMarca2(aMarc^.HI, m)
                    else
                        cantMarca2 := cantMarca2(aMarc^.HD, m);
                    end;
                end;
            end;
        end;
end;

```

d) Invoque a un módulo que reciba el árbol generado en a) i) y retorne una estructura con la información de los autos agrupados por año de fabricación.

Claramente, hay que recorrer todo el árbol de los autos ordenados por patente, y crear otra estructura donde los autos queden agrupados por año de fabricación. Pero hay dos cosas muy importantes que no debemos pasar por alto:

- 1) Cuando la consigna dice *una estructura con la información de los x **agrupados** por y criterio*, se refiere, generalmente, que debemos hacer un VECTOR DE LISTAS. Esa la forma de agruparlos: en listas, dentro de un vector.
- 2) Aunque parezca tentador usar el mismo tipo de dato que antes como dato de estas listas (o sea, el registro *auto*), pensemos que cada posición del array representa un año, entonces no hace falta guardar el año en el registro del auto. Por lo tanto, como NO HAY QUE REPETIR INFORMACIÓN, debemos crear un tipo de dato muy parecido, pero sin el año.

Acá hay una versión de estos nuevos tipos de datos que vamos a necesitar para este módulo:

```

datoListaAnios = record
    patente : integer;
    marca : string;
    color : string;
    modelo : string;
end;

```



```

listaAnios = ^nodoAnios;
nodoAnios = record
    dato : datoListaAnios;
    sig : listaAnios;
end;

vectorAnios = array [ini_anio..fin_anio] of listaAnios;

```

e) Invoque a un módulo que reciba el árbol generado en a) i) y una patente y devuelva el modelo del auto con dicha patente.

f) Invoque a un módulo que reciba el árbol generado en a) ii) y una patente y devuelva el color del auto con dicha patente.

Estos dos módulos son muy sencillos, no tienen ninguna trampa. La diferencia con otras consignas similares es que, en este caso, no podemos usar funciones, ya que deberían retornar un string, así que hay que hacerlos con procedimientos.

Analicemos ahora la siguiente consigna tomada de un examen viejo.

PlayStation Store requiere procesar las compras realizadas por sus clientes durante el año 2022.

a) Implementar un módulo que lea **compras** de videojuegos. De cada compra se lee código del videojuego, código de cliente, día y mes. La lectura finaliza con el código de cliente 0. *Se sugiere utilizar el módulo leerCompra().* Se deben retornar 2 estructuras de datos:

- i. Una estructura eficiente para la búsqueda por código de cliente.
- ii. Otra estructura que almacena la cantidad de compras realizadas en cada mes.

b) Implementar un módulo que reciba la estructura generada en a) i, un código de cliente, retorne todas las compras que realizó dicho cliente.

c) Implementar un módulo que reciba la estructura generada en a) ii y retorne la misma estructura ordenada por cantidad de mayor a menor.

NOTA: Implementar el programa principal, que invoque a los incisos a, b y c.

Por suerte, el examen ya viene con algunos pedazos de código, así que el registro de videojuego y el módulo *leerCompra()* ya están cocinados:

```

type
    sub_dias = 1..31;
    sub_meses = 1..12;

```

```

compra = record
    juego : integer;
    cliente: integer;
    dia : sub_dias;
    mes : sub_meses;
end;

procedure leerCompra(var c : compra);
begin
    c.cliente := random(50);
    if (c.cliente <> 0) then
    begin
        c.dia := 1 + random(31);
        c.mes := 1 + random(12);
        c.juego := 1000 + random(1000);
    end;
end;

```

El inciso a) i) nos está diciendo claramente: hacé un árbol de videojuegos ordenados por cliente. No está tan claro cómo espera que hagamos los nodos: podrían ser directamente el registro *compra*, o podrían ser un registro nuevo que tengan el número de cliente y una lista con todas las compras de ese cliente. Yo opté por esta última opción, pero la verdad que no estoy 100% seguro. Estoy 99 % seguro.

¿Por qué? Porque, en general, si creamos un árbol donde sus nodos están ordenados por x criterio, no tiene mucho sentido que el campo en cuestión (en este caso, el código de cliente), pueda repetirse muchas veces. En parte porque se hace difícil trabajar con una estructura así, y en parte porque se desaprovecha la eficiencia del árbol. Pensemos que si tenemos muchos nodos con el mismo código y los ingresamos todos seguidos, esos nodos se terminan ordenando como una lista (ya que irían todos hacia la derecha, no se distribuirían de forma equilibrada). Y decía que era difícil, porque si uno quiere trabajar con toda la información de algún cliente (como por ejemplo para resolver el inciso b)), la tarea se vuelve engorrosa y el algoritmo sería un poco más complicado de implementar (aunque no imposible, obvio). Además, habría que considerar si la información se ingresa ordenada o no, porque si no se ingresa ordenada, los nodos con un mismo código pueden quedar muy dispersos.

El a) ii) nos pide una estructura que almacene simplemente la cantidad de compras realizadas en cada mes. Como los meses son 12 (o sea, sabemos con certeza que todos los años tienen 12 meses, ni más ni menos), podemos usar un vector sin dimensión lógica. Ahora bien, si no leemos toda la consigna, quizás se nos ocurra hacer un vector contador del 1 al 12 de integer donde en cada elemento se almacena la cantidad vendida ese mes. Eso está muy bien, pero tiene un problema: no podemos ordenar un vector de ese estilo. Por lo tanto, como en el

inciso c) nos pide que lo ordenemos, tendremos que hacer un vector de registros, donde cada registro almacena el mes y la cantidad vendida ese mes. O sea, algo así:

```
datoVector = record
    mes : sub_meses;
    cant : integer;
end;

vectorMeses = array [1..12] of datoVector;
```

Habiendo hecho un árbol ordenado por cliente, donde cada nodo tiene la lista de las compras de ese cliente, todo lo que tenemos que hacer para resolver el inciso b) es buscar el cliente que nos interesa, y retornar esa lista con una función. ¿Por qué podemos usar una función? Porque si bien la lista conceptualmente es una estructura de datos, en términos prácticos, enviar o recibir una lista se reduce a enviar o recibir un puntero (que es un tipo de dato simple), así que por eso podemos usar una función.

Esta función, puede escribirse así:

```
function dameLasCompras (a : arbol; cli : integer) : lista;
begin
    if (a = nil) then
        dameLasCompras := nil // Si se terminó el árbol, y no encontró el cliente retorna nil
    else
        begin
            if (cli = a^.dato.cliente) then
                dameLasCompras := a^.dato.l // Si encontró el cliente, retorna su lista
            else
                begin // Si no se terminó el árbol, y no encontró el cliente, sigue buscando
                    if (cli < a^.dato.cliente) then // o bien por izquierda
                        dameLasCompras := dameLasCompras(a^.HI, cli)
                    else // o bien por derecha
                        dameLasCompras := dameLasCompras(a^.HD, cli)
                    end;
                end;
            end;
        end;
    end;
```

El inciso c) simplemente pide ordenar el vector de los meses según la cantidad de ventas. Para eso podemos usar selection sort o insertion sort.